

MAS Final Project: Agent Semantics (`agent.py`)

Complete Code Documentation

1. Scope

This chapter documents `agent.py` fully, covering:

- constants and movement indexing,
- action/direction enums,
- `Robot` state and behavior,
- movement collision rules,
- pick/drop/deliver lifecycle,
- reward/event outputs returned to environment,
- contracts with `env.py` and planner metadata.

2. Imports and Module Constants

Imports:

- `random`
- `Enum` from `enum`
- typing: `Any`, `Dict`, `Optional`, `Tuple`

Movement lookup tables:

- `MOVE_X = [0, 1, 0, -1]`
- `MOVE_Y = [-1, 0, 1, 0]`

Index mapping is directional:

- 0: UP
- 1: RIGHT
- 2: DOWN
- 3: LEFT

These arrays are used for forward-step displacement.

3. Direction Enum

`Direction(Enum)` values:

- `UP = 0`
- `RIGHT = 1`
- `DOWN = 2`
- `LEFT = 3`

This enum provides orientation state for each robot and index alignment with movement arrays.

4. Action Enum

Action(Enum) values:

- FORWARD = 0
- TURN_LEFT = 1
- TURN_RIGHT = 2
- PICK_DROP = 3
- WAIT = 4

These integer values are the runtime action protocol used by planner and environment.

5. Robot Class Overview

Robot models one mobile agent in the grid world.

Persistent fields:

- id: integer robot identity
- x, y: current grid cell
- dir: current heading (Direction)
- carrying: optional shelf dict when carrying, else None

6. Constructor (__init__)

Signature:

- __init__(self, robot_id: int, x: int, y: int) -> None

Behavior:

1. Stores identity and spawn position.
2. Initializes direction randomly:
 - self.dir = random.choice(list(Direction))
3. Initializes carrying state:
 - self.carrying = None

Implication:

- initial orientation is stochastic unless global seed is fixed.

7. Rotation Methods

7.1 turn_left

- updates direction with modular decrement:
 - (dir.value - 1) % 4

7.2 turn_right

- updates direction with modular increment:
 - `(dir.value + 1) % 4`

Both are in-place and do not change position.

8. Forward Motion (`forward`)

Signature:

- `forward(self, env: Any) -> Tuple[bool, bool]`

Return tuple:

- `moved`: position changed or not
- `bump`: blocked by boundary/robot/shelf

Flow:

1. Compute next forward cell using `_next_forward_cell`.
2. Check bounds with `_inside_bounds`:
 - if out of bounds -> `(False, True)`.
3. Check robot occupancy with `_occupied_by_robot`:
 - if occupied -> `(False, True)`.
4. Check shelf occupancy policy with `_occupied_by_shelf`:
 - if blocked -> `(False, True)`.
5. If all checks pass:
 - update robot `(x, y)`,
 - if carrying shelf, synchronize shelf coordinates,
 - return `(True, False)`.

Note:

- In current `env.py`, motion resolution is handled inside `env.step(...)`; this method remains a valid primitive API.

9. Pick/Drop Dispatcher (`pick_or_drop`)

Signature:

- `pick_or_drop(self, env: Any) -> Tuple[float, str]`

Behavior:

1. If not carrying:
 - delegate to `_pick_shelf_here`.
2. If carrying:
 - compute:
 - `carrying_requested`
 - `at_goal`
 - if carrying requested and at goal:

- delegate to `_deliver_shelf`.
- otherwise:
 - delegate to `_drop_shelf`.

Return protocol:

- `(reward_delta, event_name)`

Used directly by environment reward/event logic.

10. Position and Occupancy Helpers

10.1 `_next_forward_cell`

Computes:

- `dx = MOVE_X[self.dir.value]`
- `dy = MOVE_Y[self.dir.value]`
- returns `(self.x + dx, self.y + dy)`.

10.2 `_inside_bounds`

Static check:

- `0 <= x < env.grid_w`
- `0 <= y < env.grid_h`

10.3 `_occupied_by_robot`

Iterates all env robots except self.

Returns `True` if any other robot occupies `(x, y)`.

10.4 `_occupied_by_shelf`

This is a critical policy method.

Rules implemented:

1. Carried shelves do not block movement.
2. If this robot is already carrying something:
 - all non-carried shelves block.
3. If robot is not carrying:
 - planner-approved shelf cell for this robot may be entered.
4. Non-requested shelves always block.

Planner integration:

- `allowed_entries = getattr(env, "_planner_allowed_shelf_entries", {})`
- `allowed_pos = allowed_entries.get(self.id)`

Decision process per shelf at (x, y):

1. If shelf cell mismatch -> skip.
2. If shelf is carried -> skip (non-blocking).
3. If robot carrying -> block immediately.
4. If shelf requested and `allowed_pos == (x, y)` -> allow.
5. Else block.

If no blocking shelf matched, returns `False`.

11. Shelf Pickup (`_pick_shelf_here`)

Signature:

- `_pick_shelf_here(self, env: Any) -> Tuple[float, str]`

Flow:

1. Iterate shelves.
2. Require same cell as robot.
3. Skip already carried shelves.
4. On first valid shelf:
 - set `shelf["carried"] = True`
 - set `self.carrying = shelf`
 - reward/event:
 - requested shelf -> (5.0, "PICKED")
 - non-requested shelf -> (0.0, "PICKED")
5. If no valid shelf found:
 - return (-0.05, "NOOP")

12. Shelf Delivery (`_deliver_shelf`)

Signature:

- `_deliver_shelf(self, env: Any) -> Tuple[float, str]`

Flow:

1. If not carrying:
 - return (-0.05, "NOOP").
2. Save current carried shelf as `delivered_shelf`.
3. Remove it from `env.shelves`.
4. Build occupied set from:
 - robot cells,
 - current shelf cells,
 - goal cells.
5. Spawn replacement requested shelf:
 - `spawn_x, spawn_y = env.get_random_free_position(occupied)`
 - `next_id = max(existing_ids + [-1]) + 1`
 - append new shelf dict:

- id, x, y, carried=False, requested=True
- 6. Clear robot carrying state:
 - `self.carrying = None`
- 7. Return (20.0, "DELIVERED").

Design implication:

- successful delivery removes old shelf and replenishes workload with a fresh requested shelf.

13. Shelf Drop (`_drop_shelf`)

Signature:

- `_drop_shelf(self) -> Tuple[float, str]`

Behavior:

1. If not carrying:
 - `(-0.05, "NOOP")`.
2. Else:
 - set carried shelf "carried" to False,
 - clear `self.carrying`,
 - return `(-0.1, "DROPPED")`.

Drop does not relocate shelf; it remains at robot's current position.

14. Reward/Event Contract with Environment

`pick_or_drop` and its helpers emit tuple (`reward_delta`, `event`) with events:

- "PICKED"
- "DELIVERED"
- "DROPPED"
- "NOOP"

`env.step(...)` consumes this output to:

- aggregate rewards,
- increment delivery counters,
- append step events to logs.

15. Planner-Dependent Movement Access Control

`_occupied_by_shelf` enforces planner-level permissioning:

- only planner-assigned shelf entry is passable for empty robot.

Source of permission:

- `env._planner_allowed_shelf_entries[robot_id] = (x, y)` set by `CooperativePlanner`.

If planner fails to provide allowance:

- requested shelf cells are treated as blocked for entry.

This guarantees pickup behavior follows planner assignment, preventing opportunistic shelf capture.

16. Data Model Assumptions About `env`

`agent.py` expects environment to provide:

- `grid_w, grid_h`
- `robots` list with robot-like objects
- `shelves` list of dicts with keys:
 - `id, x, y, carried, requested`
- `GOALS` list
- `get_random_free_position(...)`
- optional `_planner_allowed_shelf_entries`

If these contracts change, agent behavior can break.

17. Determinism Notes

Deterministic behaviors:

- turning, movement checks, pickup/drop/delivery logic.

Non-deterministic behaviors:

- initial direction in constructor,
- delivery replacement shelf spawn position.

Global seed in main runtime controls reproducibility.

18. Edge Cases and Behavioral Consequences

1. Carrying non-requested shelf blocks further shelf entry and incurs environment penalty.
2. Attempting pick/drop on invalid context returns "NOOP" with small negative reward.
3. Delivery always spawns a new requested shelf, keeping task supply persistent.
4. If multiple shelves occupy same cell (should not normally happen), first matching shelf in iteration order is used.

19. Full Symbol Coverage Checklist

All symbols in `agent.py` are covered:

- `MOVE_X`
- `MOVE_Y`

- `Direction`
- `Action`
- `Robot.__init__`
- `Robot.turn_left`
- `Robot.turn_right`
- `Robot.forward`
- `Robot.pick_or_drop`
- `Robot._next_forward_cell`
- `Robot._inside_bounds`
- `Robot._occupied_by_robot`
- `Robot._occupied_by_shelf`
- `Robot._pick_shelf_here`
- `Robot._deliver_shelf`
- `Robot._drop_shelf`

20. Chapter Summary

`agent.py` defines the atomic semantics for robot actuation and object interaction.

It is intentionally compact but safety-critical: shelf blocking rules and pick/drop outcomes directly shape planner feasibility, environment rewards, and verification conflict dynamics.